

Esta clase va a ser

- grabada
a

Clase 07. PROGRAMACIÓN BACKEND

Express avanzado

Temario

06

Servidores web

- ✓ Protocolo HTTP
- ✓ Servidor con express js
- ✓ Objeto request

07

Express avanzado

- ✓ [Código de estado en HTTP](#)
- ✓ [Métodos de petición](#)

08

Router y Multer

- ✓ Express Router
- ✓ Express y middlewares
- ✓ Multer

Objetivos de la clase

- **Conocer** los status del protocolo HTTP
- **Comprender** el concepto de API REST
- **Conocer** los métodos POST, PUT, DELETE y utilizarlos con POSTMAN o ThunderClient
- **Profundizar** de manera práctica sobre los métodos POST, PUT, DELETE

CLASE N°6

Glosario

HTTP: Acrónimo para Hyper Text Transfer Protocol. Es el protocolo que nos permite comunicarnos a través de Internet

Protocolo: Conjunto de estándares y normas que deben seguirse para poder llevar a cabo una comunicación correcta.

Servidor: Sistema que permite recibir peticiones de otras computadoras y devolver una respuesta a éstas.

Módulo http: Módulo nativo de nodejs que permite programar un servidor http.

Express: Framework de desarrollo para crear servidores de una manera escalable.

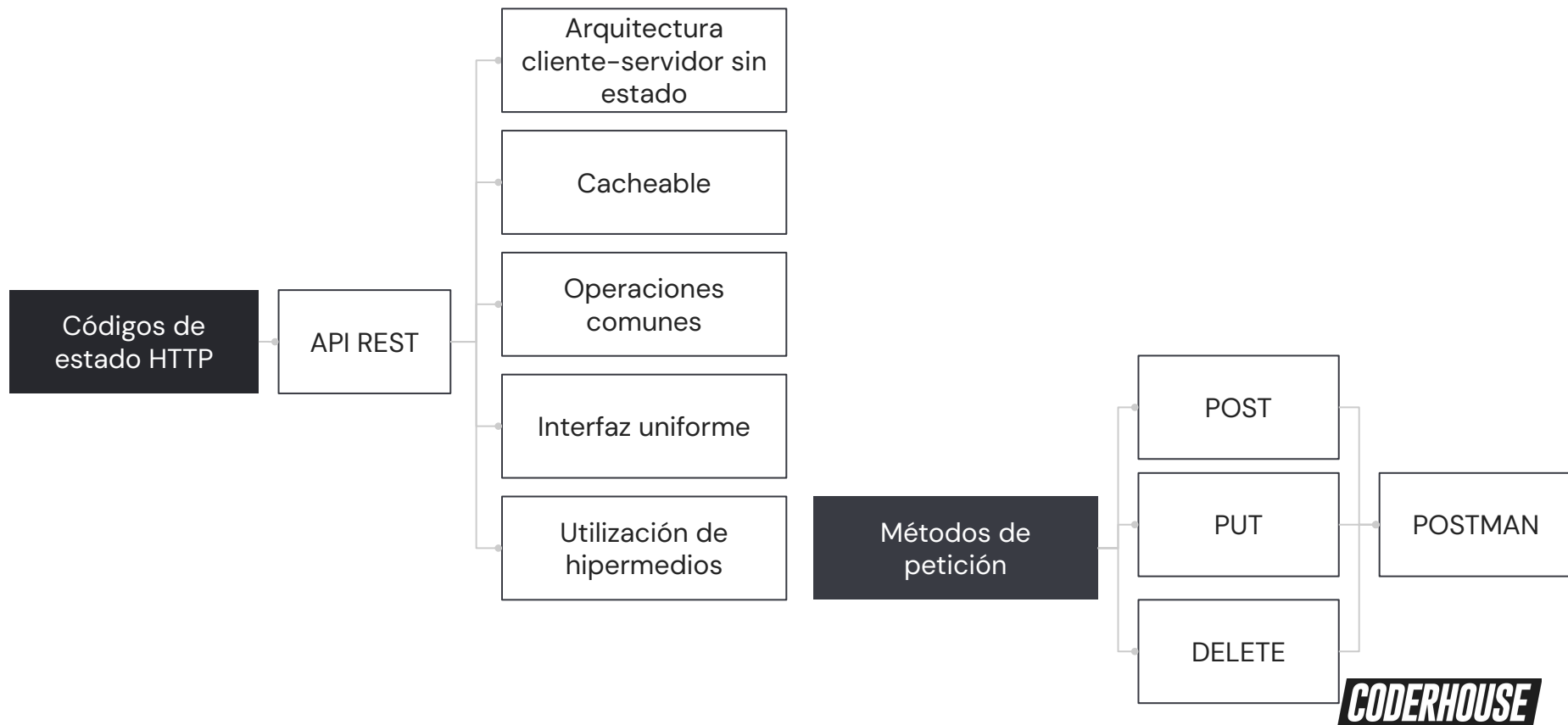
Endpoint: Punto (ruta) específico bajo el cual el servidor conectará con el servicio a proporcionar

Método GET: Método del protocolo HTTP que permite indicar al servidor que deseamos hacer una consulta de obtención de datos.

Objeto req: Objeto usado dentro de los servicios de express para poder realizar consultas más complejas. sus principales componentes son: req.method, req.url req.query, req.params, req.body



MAPA DE CONCEPTOS



Códigos de estado en HTTP

¿Qué es un código de estado?

Cuando realizamos alguna petición al servidor mediante el protocolo HTTP, el servidor debe respondernos **no sólo con información**, sino con un estado del proceso. Este es un código que nos permitirá saber cómo se encuentra el proceso, o cómo finalizó.

HTTP Status Codes



Códigos de estado HTTP

¿Cómo funciona?

Cuando el servidor responde con un código de estado, esto permite saber qué ocurrió con la consulta que estábamos haciendo, y da información al cliente sobre qué ha ocurrido.

- ✓ **1xx: Status "informativo"**
- ✓ **2xx : Status "ok"**
- ✓ **3xx: Status de redirección.**
- ✓ **4xx: Status de error de cliente.**
- ✓ **5xx: Status de error en servidor.**

HTTP STATUS CODES	
2xx Success	
200	Success / OK
3xx Redirection	
301	Permanent Redirect
302	Temporary Redirect
304	Not Modified
4xx Client Error	
401	Unauthorized Error
403	Forbidden
404	Not Found
405	Method Not Allowed
5xx Server Error	
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout

Algunos de los más importantes

- ✓ **200:** Indica que la petición se procesó correctamente. No hubo ningún tipo de inconveniente desde la consulta hasta la respuesta.
- ✓ **3xx:** Hace referencia a redirecciones, cuando un recurso se ha movido o necesitamos apuntar a otro servicio.
- ✓ **400:** Se utiliza cuando el cliente realiza alguna petición que no cumpla con las reglas de comunicación (una mala consulta, tal vez le faltó enviar un dato, o venía en un formato incorrecto).
- ✓ **401:** Se utiliza cuando el cliente no se ha identificado con el servidor bajo alguna credencial, no puede acceder al recurso

Algunos de los más importantes

- ✓ **403:** Se utiliza cuando el cliente ya se identificó, pero sus credenciales no tienen el nivel de privilegios suficiente para acceder al recurso, es el equivalente a decir "Ya qué quién eres, pero ni así estás autorizado"
- ✓ **404:** Se utiliza cuando el recurso no se ha encontrado, ya sea algún dato solicitado o incluso el endpoint mismo.
- ✓ **500:** Se utiliza cuando algo ocurrió en el servidor, no necesariamente un error del cliente, sino un error o "detalle" que no haya considerado el servidor al tratar con algún caso.

Airbnb



Shoot!

Well, this is unexpected...

Error code: 500

An error has occurred and we're working to fix the problem! We'll be up and running shortly.

If you need immediate help from our customer service team about an ongoing reservation, please [call us](#). If it isn't an urgent matter, please visit our [Help Center](#) for additional information. Thanks for your patience!

For urgent situations please [call us](#) ☎



Discord

404

¿MAGOS DETRÁS DE LAS CORTINAS?

Eso es muy de 1939. Discord lo llevan en secreto unos hámsteres robot cuánticos. Esto de la tecnología es una locura, ¿eh? De todas formas, parece que te has perdido. Aquí te sugerimos algunas páginas.

[PÁGINA DE ESTADO](#)

[@DISCORD](#)

[SOPORTE TÉCNICO](#)





¿Sabías que...?

En 1998 Se agregó un código de estado 418 con significado "I'm a teapot" (Soy una tetera) como broma del día de los inocentes para el Hyper Text Coffee Pot Control Protocol. Este estado significa que el servidor se rehúsa a realizar la tarea solicitada, porque es una tetera.

[Ver el documento oficial](#)



¡Importante!

Recuerda que tú eres el desarrollador del servidor, entonces es tu responsabilidad reconocer cuándo colocar cada código de status. Si no configuramos nuestro servidor para devolver múltiples status, entonces será mucho más difícil rastrear los problemas.

Comprendiendo una API REST

API (Application Programming Interface)

Es un conjunto de definiciones y reglas que permiten que dos equipos puedan integrarse para trabajar juntos. La mejor analogía que hay para comprender ésto es que una API funge como un “contrato” entre el front y el back.

Veamos las etapas en el siguiente Genially

La API permite entonces que se respondan preguntas como:

- ✓ ¿A qué endpoint debo apuntar para la tarea que necesito?
- ✓ ¿Qué método debo utilizar para ese recurso?
- ✓ ¿Qué información debo enviar para realizar correctamente mi petición?

El cliente necesita algo del servidor, por lo que tiene que realizar una petición (request)

Para que la petición llegue correctamente al servidor, deberá apuntar al endpoint correcto, con el método correcto, con la información correcta

El servidor recibe la petición. Si se cumplieron todas las especificaciones de la API, el procesamiento se podrá llevar a cabo con éxito

Petición

Procesamiento



Respuesta

Resultado

El cliente, al haber cumplido con lo que especificaba la API, podrá obtener su resultado satisfactoriamente y utilizarlo.

Cumplir con el contrato de la API asegura (en la mayoría de los casos) que habrá un resultado satisfactorio

REST

Ya tenemos las reglas para comunicarse, ¿Pero qué tal la estructura del mensaje? Cuando hacemos una petición o cuando recibimos una respuesta, ésta debe tener un **formato**. REST (**RE**presentational **S**tate **T**ransfer) permite definir **la estructura** que deben tener los datos para poder transferirse.

La API respondía a preguntas sobre cómo comunicarse correctamente, sin embargo, REST define cómo debe ser el cuerpo del mensaje a transmitir. (puedes llegar a hablar con el presidente si cumples con el protocolo (HTTP) y las reglas (API), pero ¿de qué nos servirá si la forma en que estructuramos nuestro mensaje (REST) no es correcta?)

Los dos formatos más importantes son **JSON** y **XML**. La utilización de la estructura dependerá de las necesidades del proyecto. Nosotros utilizaremos **JSON**. Como notarás, ¡un JSON parece un objeto! así que es mucho más amigable la sintaxis.

✓ XML

```
<factura>
  <cliente>Gomez</cliente>
  <emisor>Perez S.A.</emisor>
  <tipo>A</tipo>
  <items>
    <item>Producto 1</item>
    <item>Producto 2</item>
    <item>Producto 3</item>
  </items>
</factura>
```

✓ JSON

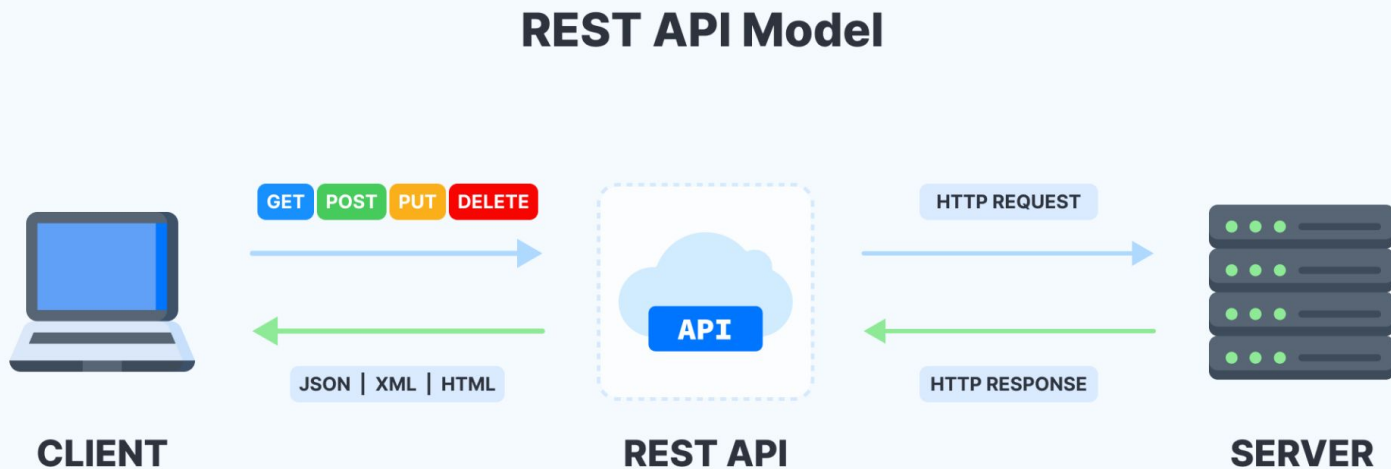
```
{
  "cliente": "Gomez",
  "emisor": "Perez S.A.",
  "tipo": "A",
  "items": [
    "Producto 1",
    "Producto 2",
    "Producto 3"
  ]
}
```

Entonces una API REST es...

Un modelo completo para tener perfectamente estipulados los protocolos, las reglas, e incluso la estructura de la información, con el fin de poder hacer un sistema de comunicación completo entre las computadoras.



Modelo de una API REST



**¿Qué características
debe tener una API REST?**

Arquitectura Cliente-Servidor sin estado

- ✓ Cada mensaje HTTP contiene toda la información necesaria para comprender la petición.
- ✓ Como resultado, ni el cliente ni el servidor necesitan recordar ningún estado de las comunicaciones entre mensajes.
- ✓ Esta restricción mantiene al **cliente** y al **servidor débilmente acoplados**: el cliente no necesita conocer los detalles de implementación del servidor y el servidor se “despreocupa” de cómo son usados los datos que envía al cliente.

Cacheable

- ✓ Debe admitir un sistema de almacenamiento en caché.
- ✓ La infraestructura de red debe soportar una caché de varios niveles.
- ✓ Este almacenamiento evita repetir varias conexiones entre el servidor y el cliente, en casos en que peticiones idénticas fueran a generar la misma respuesta.

Operaciones comunes

- ✓ Todos los recursos detrás de nuestra API deben poder ser consumidos mediante peticiones HTTP, preferentemente sus principales (POST, GET, PUT y DELETE).
- ✓ Con frecuencia estas operaciones se equiparan a las operaciones CRUD en bases de datos (en inglés: Create, Read, Update, Delete, en español: Alta, Lectura, Modificación, y Baja).
- ✓ Al tratarse de peticiones HTTP, éstas deberán devolver con sus respuestas los correspondientes códigos de estado, informando el resultado de las mismas.

Interfaz uniforme

- ✓ En un sistema REST, cada acción (más correctamente, cada recurso) debe contar con una URI (Uniform Resource Identifier), un identificador único.
- ✓ Ésta nos facilita el acceso a la información, tanto para consultarla, como para modificarla o eliminarla, pero también para compartir su ubicación exacta a terceros.

Utilización de hipermedios

- ✓ Cada vez que se hace una petición al servidor y este devuelve una respuesta, parte de la información devuelta pueden ser también hipervínculos de navegación asociada a otros recursos del cliente.
- ✓ Como resultado de esto, es posible navegar de un recurso REST a muchos otros, simplemente siguiendo enlaces sin requerir el uso de registros u otra infraestructura adicional.



Break

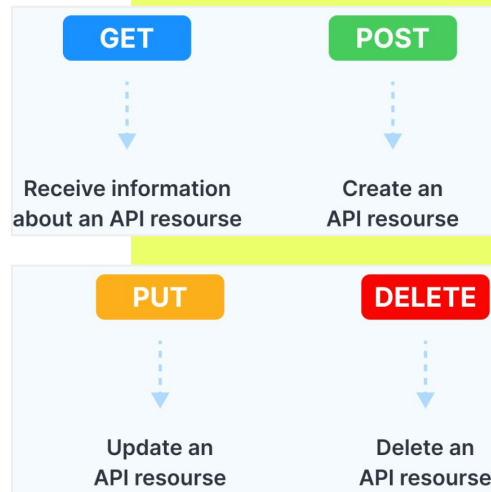
¡10 minutos y volvemos!

Métodos de petición

Métodos de petición

Un método es una definición que forma parte del protocolo HTTP, el cual nos sirve para canalizar el tipo de petición que estoy realizando sobre un cierto endpoint. De esta manera, el cliente puede llamar al mismo endpoint, **pero con diferentes métodos**, indicando qué operación quiere realizar con dicho recurso. Los principales métodos son:

- ✓ GET: Obtener un recurso
- ✓ POST: Crear o añadir un recurso
- ✓ PUT: Modificar un recurso
- ✓ DELETE: Eliminar un recurso



Principios

- ✓ Una aplicación RESTful requiere un enfoque de diseño distinto a la forma típica de pensar en un sistema: lo contrario a RPC
- ✓ RPC (Remote Procedure Calls, llamadas a procedimientos remotos) basa su funcionamiento en las operaciones que puede realizar el sistema (acciones, usualmente verbos). Ej: `getUsuario()`
- ✓ En REST, por el contrario, el énfasis se pone en los recursos (usualmente sustantivos), especialmente en los nombres que se le asigna a cada tipo de recurso. Ej. Usuarios.
- ✓ Cada funcionalidad relacionada con este recurso tendría sus propios identificadores y peticiones en HTTP.

Ejemplos sobre cómo declarar correctamente endpoints

Sí	No	¿Por qué?
GET <i>api/perritos</i> GET <i>api/perritos/:pid</i>	GET <i>api/perritos/obtener</i> GET <i>api/perritos/obtener/:pid</i>	GET ya significa "obtener" entonces no hace sentido la redundancia con el método y el endpoint.
POST <i>api/perritos</i>	POST <i>api/perritos/añadir</i>	POST ya hace referencia a crear un nuevo perrito, entonces es redundante el método y el endpoint

"api/perritos" es perfectamente funcional, sin necesidad de declarar cosas adicionales, entonces podemos reutilizar el endpoint, siempre y cuando sean diferentes sus métodos.

Ejemplos sobre cómo declarar correctamente endpoints

Sí	No	¿Por qué?
PUT <i>api/perritos/:pid</i>	PUT <i>api/perritos/actualizar/:pid</i>	PUT ya hace referencia a una actualización, hay redundancia entre el método y el endpoint
DELETE <i>api/perritos</i>	DELETE <i>api/perritos/eliminar</i>	DELETE ya significa eliminar, entonces hay redundancia entre método y endpoint.

“api/perritos” es perfectamente funcional, sin necesidad de declarar cosas adicionales, entonces podemos reutilizar el endpoint, siempre y cuando sean diferentes sus métodos.

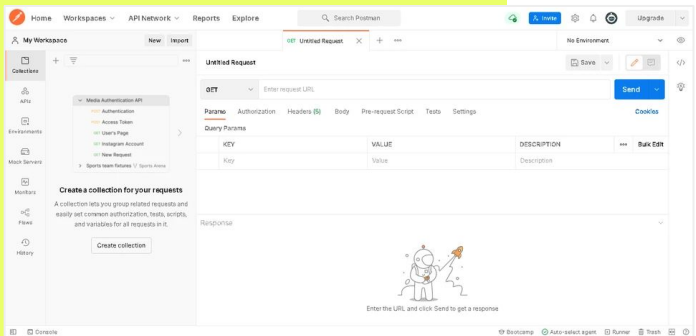
POSTMAN

POSTMAN

El problema: El navegador sólo puede enviar peticiones con método GET desde la url, (por ello es que podíamos utilizarlo desde el navegador sin problema la clase pasada), sin embargo, para poder utilizar el resto de métodos, no será posible con el navegador

La solución: POSTMAN es un software profesional que nos permitirá gestionar peticiones simulando ser un cliente. De esta manera rompemos la limitante del navegador y podemos probar todos nuestros endpoints.

[Descargar Postman](#)



Método POST

Método POST

Sirve para “crear” recursos, POST se utiliza para operaciones donde no necesitamos obtener un recurso, sino añadir uno. Algunos de los casos donde se utilizan son:

- ✓ Registrar un usuario
- ✓ Loguear un usuario
- ✓ Crear un producto
- ✓ Crear una mascota
- ✓ Crear un carrito de compra
- ✓ Enviar información para un correo electrónico.

Se apoya del recurso **req.body**, donde el body representa la información que el cliente envía para crear.

¡Importante!

Para que nuestro servidor express pueda interpretar en forma automática mensajes de tipo JSON en formato urlencoded al recibirlos, debemos indicarlo en forma explícita, agregando las siguiente líneas luego de crearlo.

```
app.use(express.json())  
app.use(express.urlencoded({ extended: true })))
```

Aclaración 🙄

`{extended:true}` precisa que el objeto `req.body` contendrá valores de cualquier tipo en lugar de solo cadenas.

¡Sin esta línea, el servidor no sabrá cómo interpretar los objetos recibidos!

Cómo utilizar un método POST

JS app.js X

EjemploExpressMethods > JS app.js > app.post('/api/user') callback

```
1 import express from 'express';
2
3 const app = express();
4 const server = app.listen(8080,()=>console.log("Listening on PORT 8080"));
5 //Primero tenemos que configurar nuestro servidor para que pueda recibir información del cliente.
6
7 app.use(express.json()) //Como indica el método, ahora el servidor podrá recibir jsons al momento de la petición
8 app.use(express.urlencoded({extended:true})) //permite que se pueda enviar información también desde la URL
9
10 let users = [] //Aquí almacenaremos los users que vayamos creando. Los crearemos a partir del método POST
11
12 //Ahora, en lugar de llamar app.get, llamaremos app.post, indicando que queremos CREAR un recurso (user)
13 app.post('/api/user',(req,res)=>{
14   let user = req.body; //Recordemos que req.body es el JSON que enviará el usuario al momento de hacer la petición
15   //Podemos validar que se cumplan ciertos campos antes de agregarlo.
16   if(!user.first_name||!user.last_name){//preguntamos si faltó el nombre o el apellido.
17     /**
18      * Ya que fue un error donde el cliente se equivocó al enviar información incompleta, el status que
19      * devolveremos será un status 400. éste lo colocaremos antes del .send como se indica más abajo.
20      */
21     return res.status(400).send({status:"error",error:"Incomplete values"})
22   }
23   //En caso de que no haya entrado al if, significa que el cliente sí envió los campos completos
24   //Procedemos a agregarlo al arreglo de users
25   users.push(user);
26   res.send({status:"success",message:"User created"}) //El status 200 viene implícito cuando no lo indicamos.
27 })
```

Probando el endpoint

Postman

File Edit View Help

Home Workspaces API Network Reports Explore

Search Postman

Invite

Upgrade

CODERHOUSE

New Import

New Collection

POST Creación de un nuev

No Environment

Collections

1 New Collection

2 POST Creación de un nuevo usuario

APIs

Environments

Mock Servers

Monitors

Flows

History

3 New Collection / Creación de un nuevo usuario

4 POST

5 Enter request URL

6 Body

7 raw

8 JSON

9

```
1 {
2   ...."first_name":"Mauricio",
3   ...."last_name":"Espinosa",
4   ...."age":25,
5   ...."email":"correoMauricio@correo.com"
6 }
```

10 Send

Params Authorization Headers (8) Body re-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

Response

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

1001

1002

1003

1004

1005

1006

1007

1008

1009

1010

1011

1012

1013

1014

1015

1016

1017

1018

1019

1020

1021

1022

1023

1024

1025

1026

1027

1028

1029

1030

1031

1032

1033

1034

1035

1036

1037

1038

1039

1040

1041

1042

1043

1044

1045

1046

1047

1048

1049

1050

1051

1052

1053

1054

1055

1056

1057

1058

1059

1060

1061

1062

1063

1064

1065

1066

1067

1068

1069

1070

1071

1072

1073

1074

1075

1076

1077

1078

1079

1080

1081

1082

1083

1084

1085

1086

1087

1088

1089

1090

1091

1092

1093

1094

1095

1096

1097

1098

1099

1100

1101

1102

1103

1104

1105

1106

1107

1108

1109

1110

1111

1112

1113

1114

1115

1116

1117

1118

1119

1120

1121

1122

1123

1124

1125

1126

1127

1128

1129

1130

1131

1132

1133

1134

1135

1136

1137

1138

1139

1140

1141

1142

1143

1144

1145

1146

1147

1148

1149

1150

1151

1152

1153

1154

1155

1156

1157

1158

1159

1160

1161

1162

1163

1164

1165

1166

1167

1168

1169

1170

1171

1172

1173

1174

1175

1176

1177

1178

1179

1180

1181

1182

1183

1184

1185

1186

1187

1188

1189

1190

1191

1192

1193

1194

1195

1196

1197

1198

1199

1200

1201

1202

1203

1204

1205

1206

1207

1208

1209

1210

1211

1212

1213

1214

1215

1216

1217

1218

1219

1220

1221

1222

1223

1224

1225

1226

1227

1228

1229

1230

1231

1232

1233

1234

1235

1236

1237

1238

1239

1240

1241

1242

1243

1244

1245

1246

1247

1248

1249

1250

1251

1252

1253

1254

1255

1256

1257

1258

1259

1260

1261

1262

1263

1264

1265

1266

1267

1268

1269

1270

1271

1272

1273

1274

1275

1276

1277

1278

1279

1280

1281

1282

1283

1284

1285

1286

1287

1288

1289

1290

1291

1292

1293

1294

1295

1296

1297

1298

1299

1300

1301

1302

1303

1304

1305

1306

1307

1308

1309

1310

1311

1312

1313

1314

1315

1316

1317

1318

1319

1320

1321

1322

1323

1324

1325

1326

1327

1328

1329

1330

1331

1332

1333

1334

1335

1336

1337

1338

1339

1340

1341

1342

1343

1344

1345

1346

1347

1348

1349

1350

1351

1352

1353

1354

1355

1356

1357

1358

1359

1360

1361

1362

1363

1364

1365

1366

1367

1368

1369

1370

1371

1372

1373

1374

1375

1376

1377

1378

1379

1380

1381

1382

1383

1384

1385

1386

1387

1388

1389

1390

1391

1392

1393

1394

1395

1396

1397

1398

1399

1400

1401

1402

1403

1404

1405

1406

1407

1408

1409

1410

1411

1412

1413

1414

1415

1416

1417

1418

1419

1420

1421

1422

1423

1424

1425

1426

1427

1428

1429

1430

1431

1432

1433

1434

1435

1436

1437

1438

1439

1440

1441

1442

1443

1444

1445

1446

1447

1448

1449

1450

1451

1452

1453

1454

1455

1456

1457

1458

1459

1460

1461

1462

1463

1464

1465

1466

1467

146

Paso a paso

1

Crear una colección

Explicar que es el equivalente de un proyecto, pero para pruebas de endpoints

2

Crear un request

Indicar que viva dentro de la colección, para mantener el orden en nuestro proyecto

3

Agregar un nombre a nuestro request

Indicamos un nombre que sea útil para reconocer la operación que realiza dicho request.

4

Cambiamos el MÉTODO

Crucial para tener claro que estamos trabajando con múltiples métodos

Paso a paso

5

Ingresamos a la URL

Endpoint a utilizar, con el fin de que coincida con el declarado en el servidor.

6

Seleccionamos la pestaña "body"

Para ingresar contenido para enviar al servidor

7

Colocamos el modo raw

Indicando que el cuerpo se creará from scratch

8

JSON

Seleccionamos la opción "JSON", ya que es la estructura que utilizaremos.

Paso a paso

9

Especificamos

El JSON a enviar

10

Send

Click al botón SEND

11

**¿Cuáles son los
resultados?**

En el panel
visualizaremos los
resultados.

Postman

File Edit View Help

Home Workspaces API Network Reports Explore

Search Postman

Invite Upgrade

CODERHOUSE New Import New Collection POST Creación de un nuevo usuario No Environment

Collections + New Collection POST Creación de un nuevo usuario

APIs Environments Mock Servers Monitors Flows History

Ejemplo probando el endpoint con datos correctos

New Collection / Creación de un nuevo usuario Save

POST http://localhost:8080/api/user Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   ... "first_name": "Mauricio",
3   ... "last_name": "Espínosa",
4   ... "age": 25,
5   ... "email": "correoMauricio@correo.com"
6 }
```

Body Cookies Headers (7) Test Results 200 OK 115 ms 280 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "status": "success",
3   "message": "User created"
4 }
```

Find and Replace Console Cookies Capture requests Bootcamp Runner Trash

Ejemplo probando el endpoint sin el campo "first_name" el cual es inválido. Notemos que ahora el status es 400 y el código es de error.

New Collection / Creación de un nuevo usuario

POST http://localhost:8080/api/user

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

☐ none
 ☐ form-data
 ☐ x-www-form-urlencoded
 ☒ raw
 ☐ binary
 ☐ GraphQL
 ☒ JSON

```
1 {
2   ... "last_name": "Espinosa",
3   ... "age": 25,
4   ... "email": "correoMauricio@correo.com"
5 }
```

Body Cookies Headers (7) Test Results

400 Bad Request 8 ms 290 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "status": "error",
3   "error": "Incomplete values"
4 }
```

Método PUT

Ya vimos cómo crear un recurso, ahora
¿Cómo podríamos modificar dicho recurso?

El método PUT sirve para esto.

Para poder trabajar con PUT, no sólo
enviamos el body en el request, sino que
además mandamos por params el id, nombre,
o cualquier identificador para que el servidor
sepa qué recurso específicamente debe
actualizar.

Hay dos formas de actualizar un recurso:
actualizar sólo los campos requeridos, o bien
mandar a actualizar el objeto completo,
ambas formas son válidas cuando hablamos
de actualización, y dependerá del contexto.

Cómo se ve una actualización básica utilizando el método PUT

```
JS app.js X
EjemploExpressMethods > JS app.js > app.post('/api/user') callback
1  import express from 'express';
2
3  const app = express();
4  const server = app.listen(8080,()=>console.log("Listening on PORT 8080"));
5  //Primero tenemos que configurar nuestro servidor para que pueda recibir información del cliente.
6
7  app.use(express.json()) //Como indica el método, ahora el servidor podrá recibir jsons al momento de la petición
8  app.use(express.urlencoded({extended:true})) //permite que se pueda enviar información también desde la URL
9
10 let users = [] //Aquí almacenaremos los users que vayamos creando. Los crearemos a partir del método POST
11
12 //Ahora, en lugar de llamar app.get, llamaremos app.post, indicando que queremos CREAR un recurso (user)
13 app.post('/api/user',(req,res)=>{
14   let user = req.body; //Recordemos que req.body es el JSON que enviará el usuario al momento de hacer la petición
15   //Podemos validar que se cumplan ciertos campos antes de agregarlo.
16   if(!user.first_name||!user.last_name){//preguntamos si faltó el nombre o el apellido.
17     /**
18      * Ya que fue un error donde el cliente se equivocó al enviar información incompleta, el status que
19      * devolveremos será un status 400. éste lo colocaremos antes del .send como se indica más abajo.
20      */
21     return res.status(400).send({status:"error",error:"Incomplete values"})
22   }
23   //En caso de que no haya entrado al if, significa que el cliente sí envió los campos completos
24   //Procedemos a agregarlo al arreglo de users
25   users.push(user);
26   res.send({status:"success",message:"User created"}) //El status 200 viene implícito cuando no lo indicamos.
27 })
```

Método DELETE

Como bien lo indica el nombre, este método lo utilizamos cuando queremos eliminar algún recurso. Aquí no es necesario enviar nada desde el body, sin embargo, sí es importante indicar en el req.params el identificador para que el servidor reconozca qué recurso debe eliminar.

Cómo se ve la utilización del método DELETE

JS app.js ×

EjemploExpressMethods > JS app.js > app.delete('/api/user/:name') callback > users.filter() callback

```
56     }
57   })
58
59   app.delete('/api/user/:name', (req, res) => {
60     let name = req.params.name;
61     let currentLength = users.length;
62     users = users.filter(user => user.first_name !== name);
63     if (users.length === currentLength) { // Si la longitud del arreglo es igual, entonces no se eliminó nada
64       return res.status(404).send({ status: "error", error: "User not found" });
65     }
66     res.send({ status: "success", message: "User deleted" });
67   })
```



Ejemplo en vivo: Integrando todos los métodos

- ✓ Se agregará al código de explicación un método GET al mismo endpoint, con el fin de completar los 4 métodos principales.
- ✓ Se realizará un flujo completo con POSTMAN donde podremos ver trabajando a todos los endpoints en conjunto, revisando,



Servidor con GET, POST, PUT, DELETE

Servidor con integración de todos los métodos

Duración: 20 min

Se trabajará de forma individual en la sala general



ACTIVIDAD EN CLASE

Servidor con GET, POST, PUT, DELETE

Dada la frase: “Frase inicial”, realizar una aplicación que contenga un servidor en express, el cual cuente con los siguientes métodos:

- ✓ GET '/api/frase': devuelve un objeto que como campo 'frase' contenga la frase completa
- ✓ GET '/api/palabras/:pos': devuelve un objeto que como campo 'buscada' contenga la palabra hallada en la frase en la posición dada (considerar que la primera palabra es la #1).



ACTIVIDAD EN CLASE

- ✓ POST '/api/palabras': recibe un objeto con una palabra bajo el campo 'palabra' y la agrega al final de la frase. Devuelve un objeto que como campo 'agregada' contenga la palabra agregada, y en el campo 'pos' la posición en que se agregó dicha palabra.
- ✓ PUT '/api/palabras/:pos': recibe un objeto con una palabra bajo el campo 'palabra' y reemplaza en la frase aquella hallada en la posición dada. Devuelve un objeto que como campo 'actualizada' contenga la nueva palabra, y en el campo 'anterior' la anterior.
- ✓ DELETE '/api/palabras/:pos': elimina una palabra en la frase, según la posición dada (considerar que la primera palabra tiene posición #1).
- ✓ Utilizar POSTMAN para probar funcionalidad



¿Estamos repitiendo demasiado?

Notamos que podemos utilizar el mismo endpoint para el mismo tipo de recurso, siempre y cuando haya diferentes métodos.

Si se repiten, ¿habrá forma de agrupar estos endpoints para tener un código mucho más limpio?

¿Preguntas?

Resumen de la clase hoy

- ✓ Protocolo HTTP
- ✓ API, REST, API REST
- ✓ Métodos POST, PUT, DELETE
- ✓ POSTMAN

Opina y valora
esta clase

Muchas gracias.

#DemocratizandoLaEducación